Data Mining Technique Assignment 1

Group 45: Antal Deurloo (2706710), Yaqi Duan(2839784), and Zheng Xia(2906182)

April 2026

Abstract. This report presents an analysis of a longitudinal smartphone dataset collected from 27 users over approximately 113 days. We address data quality issues, perform missing-value imputation, engineer features, and apply both classification and regression models to predict mood. We compare temporal and non-temporal approaches, finding that feature-engineered tabular models outperform LSTM-based methods on this dataset. We also examine generalized association rules as an additional analysis technique.

Keywords: Mood prediction · Feature engineering · Time series · LSTM · Gradient boosting · Association rules

1 Introduction

1.1 Dataset Overview

The dataset contains 376,912 records collected from 27 users over roughly 113 days (17 February to 9 June 2014). Although user IDs range from `AS14.01` to `AS14.33`, only 27 are present, suggesting dropout or exclusions during the study.

The data is stored in long format: each row represents one measurement of one variable for one user at one timestamp, with columns `id`, `time`, `variable`, and `value`. This format is used because the 19 variables are sampled at very different frequencies. The variables fall into three groups:

- **Self-reports**: `mood` (1–10), `circumplex.valence` and `circumplex.arousal` (both −2 to +2).
- **Phone usage durations** (seconds): `screen` and 11 `appCat.*` categories.
- **Behavioral events**: `activity` (0–1), `call`, and `sms`.

Table 1 summarizes all 19 variables, covering the ranges, distributions, and missing values requested in the task description.

Table 1. Summary statistics for all 19 variables in the dataset.

Variable	n	n_miss	min	max	mean	med	std
screen	96578	0	0.04	9867.01	75.34	20.04	253.82
appCat.builtin	91288	0	-82798.87	33960.25	18.54	4.04	415.99
appCat.communication	74276	0	0.01	9830.78	43.34	16.23	128.91
appCat.entertainment	27125	0	-0.01	32148.68	37.58	3.39	262.96
activity	22965	0	0.00	1.00	0.12	0.02	0.19
appCat.social	19145	0	0.09	30000.91	72.40	28.47	261.55
appCat.other	7650	0	0.01	3892.04	25.81	10.03	112.78
appCat.office	5642	0	0.00	32708.82	22.58	3.11	449.60
mood	5641	0	1.00	10.00	6.99	7.00	1.03
circumplex.arousal	5597	46	-2.00	2.00	-0.10	0.00	1.05
circumplex.valence	5487	156	-2.00	2.00	0.69	1.00	0.67
call	5239	0	1.00	1.00	1.00	1.00	0.00
appCat.travel	2846	0	0.08	10452.62	45.73	18.14	246.11
appCat.utilities	2487	0	0.25	1802.65	18.54	8.03	60.96
sms	1798	0	1.00	1.00	1.00	1.00	0.00
appCat.unknown	939	0	0.11	2239.94	45.55	17.19	119.40
appCat.finance	939	0	0.13	355.51	21.76	8.03	39.22
appCat.game	813	0	1.00	5491.79	128.39	43.17	327.15
appCat.weather	255	0	1.00	344.86	20.15	15.12	24.94

A few things stand out from the table. First, `appCat.builtin` and `appCat.entertainment` contain negative duration values (as low as −82,798 seconds), which is not physically meaningful and should be addressed during cleaning. Second, most app-usage variables have a mean much larger than their median (e.g. `screen`: mean 75.3, median 20.0), indicating heavy-tailed distributions with a small number of very long sessions.

Regarding missing values, only 202 cells (0.05%) are explicitly missing, all in `circumplex.valence` (156) and `circumplex.arousal` (46). This pattern suggests these fields were optional in the self-report interface, while `mood` was required. However, long-format data also hides implicit missingness: days on which a user did not complete a self-report produce no rows at all. The effective missingness of mood is therefore higher than the table suggests and should be re-evaluated at the user-day level.

1.2 Visual Exploration

We focus on three plots that each highlight a different aspect of the data: its structure across users, the shape of the target variable, and the relationships between variables.

Records per User by Variable Family. Figure 1 shows the number of records contributed by each user, broken down into three variable families: self-reports, phone usage, and behavioral events. Two things stand out. First, the total contribution is highly uneven: the most active users provide several times more records than the least active, extending the dropout pattern suggested by the missing user IDs. Second, phone-usage records dominate for every user, while self-reports make up only a small slice. This means the target variable is much sparser than the raw record count suggests, and any per-user analysis must account for these imbalances.

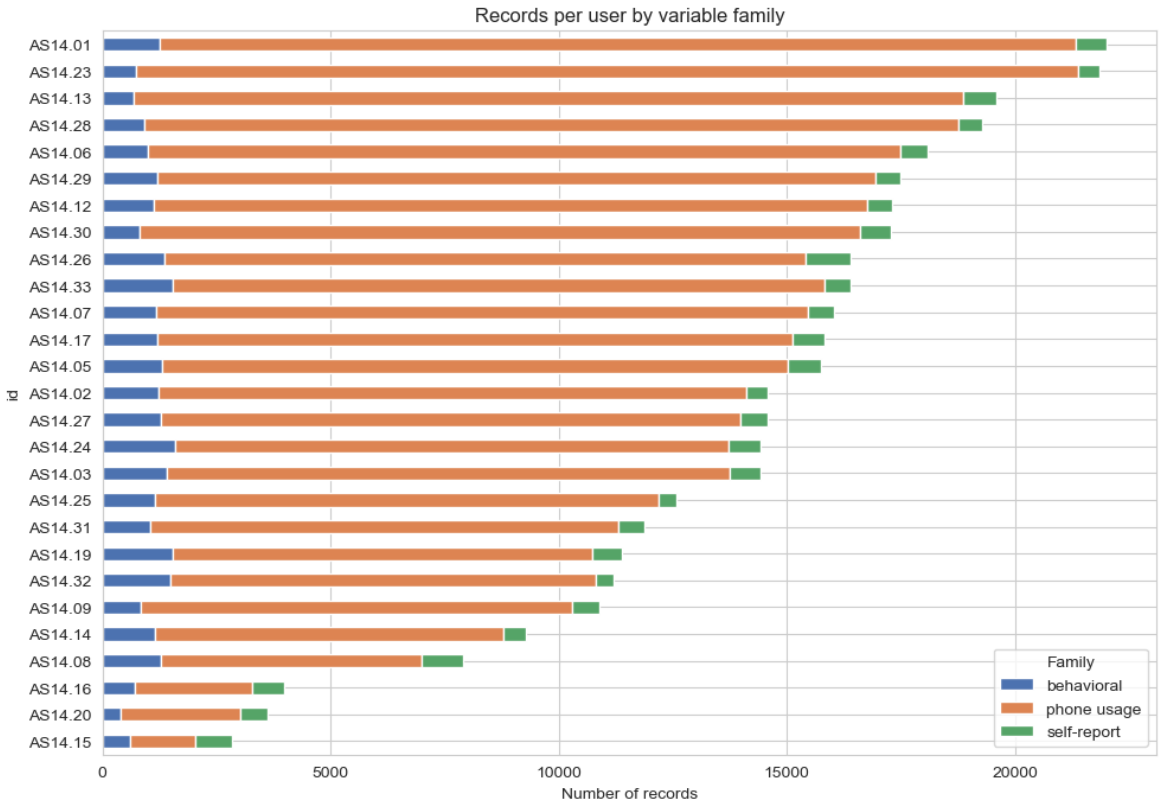

Fig. 1. Records per user, split by variable family.

Distribution of Mood. Figure 2 shows the overall mood distribution (left) and mood per user (right). The overall distribution is concentrated around 7 and rarely drops below 5. The per-user boxplot reveals that individual medians differ by more than a full point on the 1–10 scale, meaning each user has their own baseline. This suggests that per-user normalization may be useful for later modelling. A mild weekend effect is also present (Saturday mean 7.2 vs. weekday mean 6.9), but no clear within-day pattern emerges once low-sample night-time hours are excluded.

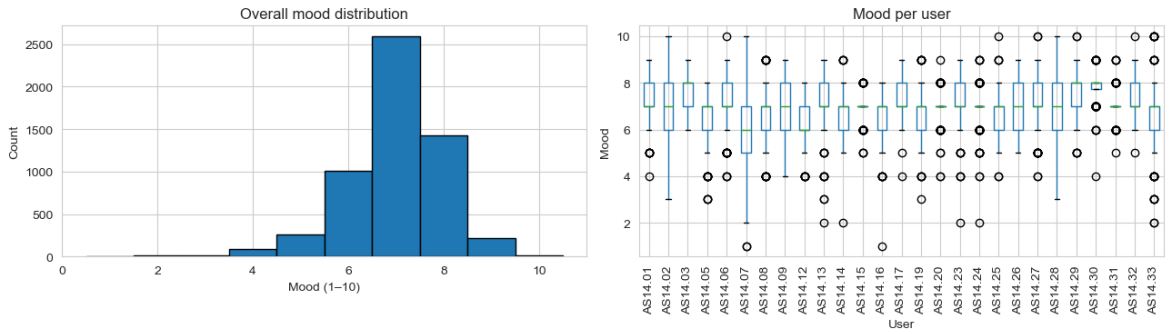

Fig. 2. Left: overall mood distribution. Right: mood per user.

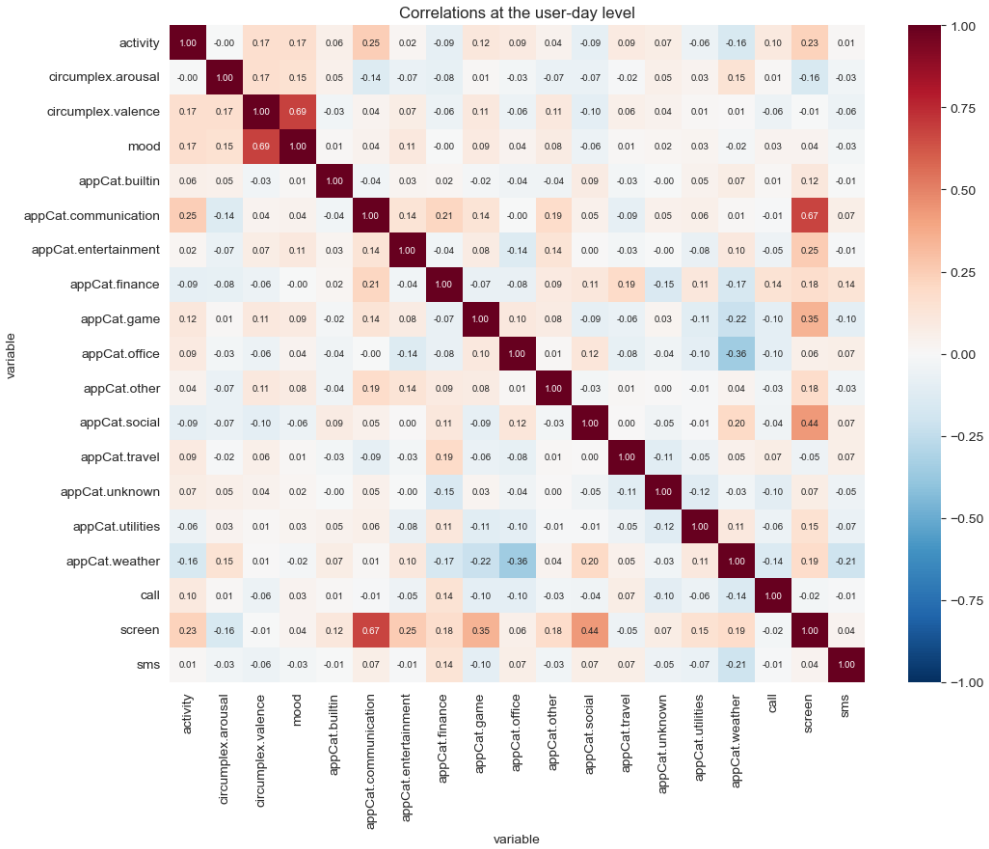

Fig. 3. Pearson correlations between all variables at the user-day level.

Relationships Between Variables. Figure 3 shows Pearson correlations between all variables aggregated to the user-day level. There are four main observations:

- Mood is strongly correlated with `circumplex.valence` ($r = 0.69$) but only weakly with `circumplex.arousal` ($r = 0.15$).
- `screen` is strongly correlated with `appCat.communication` ($r = 0.67$) and `appCat.social` ($r = 0.44$), showing that screen time is mostly driven by these two categories.
- Physical `activity` is weakly but positively correlated with mood ($r = 0.17$), matching the intuitive idea that being more active goes with better mood.
- No individual app category correlates with mood above $|0.12|$. Raw usage durations alone are therefore unlikely to be strong predictors of mood, and engineered features will probably be needed in later tasks.

2 Data Cleaning and Missing-Value Imputation

The dataset consists of longitudinal smartphone-based measurements collected at irregular timestamps for multiple subjects. Three main data-quality issues were observed: implausible values, asynchronous measurement times across variables, and substantial missingness in the target variables. To address these issues, the preprocessing procedure was organized into two stages: removal of incorrect values and imputation of missing values.

2.1 Removal of Incorrect and Extreme Values

Methods and Rationale. Incorrect values were identified using domain-based validity constraints. For each variable, an admissible range was specified according to its scale or substantive meaning. Observations outside these admissible ranges were removed.

A temporal feasibility check was used for duration-like variables. For each subject-variable series, observations were ordered by time and the time interval between consecutive observations was computed. If a recorded duration exceeded the available elapsed time since the previous observation, that value was treated as infeasible and removed.

This approach is preferable to conventional distribution-based outlier rules, such as the interquartile range (IQR) criterion, for two reasons. First, the observations are recorded at unequal time intervals, so the scale of plausible usage naturally depends on the length of the measurement window. Second, the empirical distribution of usage is not directly comparable across rows, because each row may correspond to a different amount of available time. By contrast, the temporal feasibility check is based on a simple physical constraint: a recorded duration cannot exceed the amount of time available in the interval in which it was measured.

Results. Table 2 reports the total number of observations removed by each rule. Table 3 reports the variable-level distribution of flagged observations.

Table 2. Summary of flagged observations by cleaning rule.

Cleaning rule	Number flagged	Percentage
Fixed-range violations	4	0.001%
Temporal feasibility violations	45,995	12.203%
Total removed	45,999	12.204%

Table 3. Number of flagged observations by variable and cleaning rule.

Variable	Fixed-range	Temporal feasibility
appCat.builtin	3	11,592
appCat.entertainment	1	2,767
appCat.communication	0	13,765
screen	0	12,823
appCat.social	0	3,277
appCat.office	0	369
appCat.travel	0	351
appCat.finance	0	306
appCat.other	0	301
appCat.utilities	0	176
appCat.game	0	135
appCat.unknown	0	123
appCat.weather	0	10

2.2 Preparation for Imputation

After cleaning, the dataset was transformed from long format to wide format. Feature values were aligned to the timestamps of the target variables, `circumplex.arousal` and `circumplex.valence`. For each target timestamp, recent feature information was summarized over a backward-looking 24-hour window. Count-like variables such as `call`, `sms`, and app-category usage were aggregated using sums, whereas continuous variables such as `activity` were aggregated using means. Feature-availability indicators were also constructed to record whether a feature had been observed within the window.

2.3 Missing-Value Imputation

Methods. After cleaning, 202 missing target values remained: 156 in `circumplex.valence` and 46 in `circumplex.arousal`. The missingness mechanism could not be identified directly from the data; in health-related time series, missingness is often not MCAR [9].

Two imputation methods were selected: time-based linear interpolation and BRITS (Bidirectional Recurrent Imputation for Time Series). Linear interpolation was used as a strong baseline because recent benchmarking found that it achieved the lowest RMSE across MCAR, MAR, and NMAR settings [9]. BRITS was used as the model-based alternative because it is specifically designed for multivariate time-series imputation and models temporal structure bidirectionally without relying on linear assumptions [3]. The model was fitted using the training set only, avoiding leakage across data splits.

Results. The diagnostic analyses consistently indicated that linear interpolation was more appropriate than BRITS for the present dataset. As shown in Table 4, BRITS produced values strongly concentrated around zero, whereas linearly imputed values remained closer to observed means and standard deviations. The gap-length sensitivity analysis showed the same pattern: even for short gaps, BRITS produced much less variable imputations than linear interpolation (see Table 5). Overall, these results suggest that linear interpolation better preserves the empirical structure of the target variables and is therefore the more defensible primary imputation method in this setting.

Table 4. Distributional comparison of observed and imputed target values in the training set.

Variable	Group	n	Mean	SD	Median
circumplex.arousal	observed	3668	-0.166	1.040	0.000
circumplex.arousal	linear	4	-0.098	0.820	-0.052
circumplex.arousal	BRITS	4	0.018	0.074	-0.000
circumplex.valence	observed	3659	0.637	0.701	1.000
circumplex.valence	linear	13	0.462	0.453	0.375
circumplex.valence	BRITS	13	0.000	0.058	0.010

Table 5. Gap-length sensitivity summary for the training set.

Variable	Gap	n	Lin. mean	Lin. SD	BRITS mean	BRITS SD
circumplex.arousal	1	4	-0.098	0.820	0.018	0.074
circumplex.valence	1	11	0.546	0.443	0.001	0.063
circumplex.valence	2	2	0.000	0.000	-0.008	0.028

2.4 Feature Engineering

In order to enable the application of standard machine learning algorithms we needed to transform the original time-series dataset into an instance-based dataset, where historical observations are aggregated into fixed-length feature vectors. Each instance represents a user at a specific day t, where the target variable is defined as the average mood of that day, and the features are constructed from values observed in the preceding time window.

A sliding window of three days was used to capture short-term temporal dependencies. For the main behavioral variables, the features were created using mean and the standard deviation as aggregation functions. More precisely, the features *mood_mean_3d* and *mood_std_3d* were included to capture the average level and variability of mood over recent days. Similarly, *activity_mean_3d* and *screen_mean_3d* were used to represent recent behavioral patterns. These aggregated features are designed to reduce noise in the data while preserving meaningful trends. Such aggregation strategies are common to use in similar time-series forecasting to improve predictive performance [1].

In addition to aggregated features, certain lagged variables were included explicitly. Specifically, *mood_lag1*, *mood_lag2*, and *activity_lag1* were used as predictors, where *variable_lagx* means the xth last value recorded for this variable before the target day. These features directly capture temporal dependencies and autocorrelation in the data, which is particularly relevant for psychological variables such as mood that evolve gradually over time [2].

To capture higher-level behavioral patterns, app usage variables were grouped into broader categories. The features *social_usage_mean_3d* and *entertainment_usage_mean_3d* represent aggregated usage of social and entertainment applications over the recent time window. This grouping reduces dimensionality and allows the model to focus on general behavioral patterns rather than individual app categories.

Emotional state variables were introduced in the form of the arousal and valence variables. Their average values over the time window were included as *circumplex.arousal_mean_3d* and *circumplex.valence_mean_3d*. In addition, an interaction term *arousal_x_valence* was constructed to capture the combined effect of these dimensions, reflecting the idea that emotional states are jointly determined rather than independently.

To account for behavioral stability, variability measures were included using *activity_std_5d* and *screen_std_5d*, which represent the standard deviation of activity and screen time over a longer five-day window. The purpose of these features is to capture irregularities in the daily routines, which may be an indication for changes in the mood of the subjects.

Finally, two more features were added to take into account which day of the week the target day is, as this is likely to have a substantial influence on that days mood. Because of the cyclical nature

of weeks, it would not be accurate to simply number the days from 0 to 6 and use that as a feature. If Monday is 0 and Sunday is 6 then numerically these would be far apart even though they are actually very close. This is why the following sine and cosine transformation was performed to create the features of

$$\text{day_sin} = \sin\left(\tfrac{2\pi d}{7}\right), \quad \text{day_cos} = \cos\left(\tfrac{2\pi d}{7}\right), \quad d \in \{0, \ldots, 6\} \tag{1}$$

The full set of 16 engineered features is listed in Table 6.

Table 6. Overview of engineered features.

Category	Features
Lag features	mood_lag1, mood_lag2, activity_lag1
Short-term aggregation (3d)	mood_mean_3d, mood_std_3d, activity_mean_3d, screen_mean_3d
	social_usage_mean_3d, entertainment_usage_mean_3d
	circumplex.arousal_mean_3d, circumplex.valence_mean_3d
Interaction	arousal_x_valence
Long-term variability (5d)	activity_std_5d, screen_std_5d
Temporal context	day_sin, day_cos

3 Classification

3.1 Prediction target and evaluation setup

For the classification task, the numerical prediction was formulated as a discrete setting. The target variable was defined as the subject-level average mood on the next day, rounded to the nearest integer and restricted to the original scale $[1, 10]$. This results in a classification problem with 10 classes.

The data was split at the subject level into training, validation, and test sets using a 70% / 15% / 15% ratio. This ensures that all observations from a given subject are assigned to the same split, avoiding leakage due to temporal dependencies within subjects. The validation set was used for model selection and hyperparameter tuning, while the test set was used only for final evaluation.

Because the class distribution is imbalanced, multiple evaluation metrics were considered. In addition to accuracy, both macro-averaged F1 and weighted F1 were reported. Macro F1 was treated as the main metric, since it gives equal weight to all classes and is therefore more informative in the presence of imbalance.

3.2 Compared classification algorithms

HistGradientBoosting classifier As the non-temporal classification model, a HistGradientBoostingClassifier was used. The model was trained on the feature-engineered dataset constructed in Task 1C, where each instance represents a subject-day and temporal information is encoded through lagged variables, aggregated features, and interaction terms.

In this representation, temporal dependencies are captured explicitly through the engineered predictors rather than learned from sequential input. Gradient boosting was selected because it can model nonlinear relationships and interactions between features while remaining effective on tabular data.

The model hyperparameters were tuned on the validation set. The final configuration used a learning rate of 0.04, a maximum tree depth of 8, and 700 iterations.

LSTM classifier As the temporal model, a Long Short-Term Memory (LSTM) network was used. In contrast to the boosting model, the LSTM was trained on the daily-aggregated dataset without feature engineering. Each subject-day is represented by a vector of daily summaries, and sequences of length 7 were used as input.

The model architecture consisted of a single LSTM layer with 32 hidden units, followed by a dropout layer with rate 0.2 and a fully connected output layer. The model was trained using the Adam optimizer with cross-entropy loss. Early stopping based on validation accuracy was applied to prevent overfitting.

3.3 Results

The two approaches produced clearly different results. The HistGradientBoosting classifier achieved a test accuracy of 0.821, with a macro F1 score of 0.564 and a weighted F1 score of 0.811. Performance across subjects was relatively stable, although some variation was observed.

The LSTM classifier performed substantially worse. While it achieved a validation accuracy of 0.559, its test accuracy was 0.609, with a macro F1 score of 0.155 and a weighted F1 score of 0.496.

The model based on feature-engineered predictors performs better than the temporal neural network. In the present dataset, the relevant temporal structure appears to be captured effectively through aggregation and lag features, whereas the LSTM does not manage to learn this structure directly from the sequences.

A possible explanation is that the dataset is relatively small once split at the subject level, which limits the effectiveness of the LSTM. In addition, the imbalance in the class distribution makes the classification task more difficult, particularly for the minority classes, which is reflected in the low macro F1 score of the LSTM.

Overall, the results suggest that the feature-engineered representation is more suitable for this task, and that the boosting model provides more reliable predictions.

Table 7. Classification performance on the test set.

Model	**Accuracy**	**Macro F1**	**Weighted F1**
HistGradientBoosting	0.821	0.564	0.811
LSTM	0.609	0.155	0.496

3.4 Winning Classification Algorithms

The *HMS – Harmful Brain Activity Classification* competition was hosted on Kaggle by Harvard Medical School between January and April 2024, and attracted 2767 teams ([4]). The data contains about 106,800 EEG segments of 50 seconds each, recorded from ICU patients. The task is a 6-class classification problem: each segment must be labeled as one of six brain activity patterns (Seizure, LPD, GPD, LRDA, GRDA, or Other). An important detail is that the labels are not single classes but vote counts from multiple neurologists, who often disagreed on the same segment. The evaluation metric is therefore the Kullback–Leibler divergence between the predicted and the true label distributions.

The competition was won by Team Sony ([8]). Their solution is not a single model but an ensemble of several strong single models, each built by a different team member with a different idea. The shared framework is the same across all four members, while the internal representation of the EEG signal differs.

The shared framework has two main elements. First, all four members convert the 50-second EEG signal into a 2D image and feed it into a modern image backbone from the *timm* library, such as ConvNeXt, InceptionNeXt, MaxViT, or Swin Transformer v2. The image is always built from the bipolar montage of the raw EEG, but the way it is turned into 2D differs per member: yamash simply arranges the raw waveform into an image, while suguuuuu and kfuji each apply a continuous wavelet transform (with different mother wavelets) and stack the resulting scalograms vertically. Muku uses both a time-frequency transform (superlet or STFT) and a set of learned temporal feature maps from a 1D convolution, combining multiple views within a single model. This diversity is important, because the individual models then make different kinds of mistakes and ensemble well.

Second, all members use the same two-stage training schedule. In stage 1, the model is trained on all samples with more than one vote, using a learning rate of 1e-3. In stage 2, training continues only on samples with at least ten votes, with the learning rate dropped to 1e-4. The idea is that the full set of labels is noisy because many samples have few or disagreeing annotators, so the first stage gives the model broad exposure while the second stage sharpens it on the cleaner labels.

Several smaller choices separate their solution from more standard approaches. They replace the final softmax with *entmax*, a function that can output exact zeros. This matches the training labels better, since many ground-truth distributions contain zero-probability classes, while softmax can never output zero. This single change improved the leaderboard score by about 0.004. Muku additionally applies label smoothing before normalization: adding a small offset of 0.02 to each vote count before converting to a probability distribution, which implicitly applies stronger smoothing to low-confidence labels (those with few votes) than to high-confidence ones. For ensembling, instead of simple averaging, yamash fits a non-negative linear regression per class, which gave a better correlation between local cross-validation and the public leaderboard.

What separates the winning solution from typical baselines is not a single novel component but the combination of many careful choices around the specific structure of this dataset: the noisy vote-distribution labels, the non-stationary nature of EEG, and the sensitivity of KL divergence to overconfident predictions. Their advantages include using four diverse representations instead of a single one, training in two stages instead of one, outputting sparse distributions via entmax instead of dense softmax, smoothing low-confidence labels more than high-confidence ones, and learning ensemble weights instead of averaging.

4 Association Rules

4.1 Generalized Association Rules

APRIORI treats every item as a separate symbol, so similar products such as *Pizza Margherita* and *Pizza Quattro Formaggio* are counted independently. In large catalogs, each specific item has low support, and useful patterns like "buying pizza" are missed.

Srikant and Agrawal address this with *Generalized Association Rules* ([6]). The input includes a *taxonomy* (an is-a hierarchy) over the items, for example *Margherita* → *Pizza* → *Food*. Rules can then contain items from any level of the hierarchy. A rule like $\{Pizza\} \Rightarrow \{Milk\}$ can be found even when no specific pizza variant alone passes the support threshold.

The basic idea is to extend each transaction with all ancestors of its items: a basket with *Margherita* is also treated as containing *Pizza* and *Food*. A modified APRIORI is then run on this extended data. Two extra steps are added during candidate generation. First, an item is never paired with its own ancestor, since a rule like $\{Margherita\} \Rightarrow \{Pizza\}$ is trivially true by the taxonomy itself. Second, counting uses an optimization: once a specific item has been counted, its ancestors' counts can be updated at the same time, which avoids redundant passes over the data.

Because many rules at different taxonomy levels can describe the same pattern, the authors also define when a rule is *interesting*. A rule $X \Rightarrow Y$ is pruned if a more general rule $X' \Rightarrow Y'$ (obtained by replacing items with their ancestors) already has similar support and confidence. The intuition is that if the general rule already predicts the specific rule's statistics within a small factor, then the specific rule adds no new information and can be removed. This leaves a smaller, non-redundant set of rules that span multiple levels of abstraction.

4.2 Pros and cons

The main benefit is that generalization recovers patterns hidden at the item level. High-level rules have much higher support, so real associations that would otherwise fall below the threshold become visible. The resulting rules are also easier to use in practice: a retailer usually cares more about "pizza drives cola sales" than about a single brand-specific rule. Another advantage is that the method can mix levels freely, so it can detect that a specific item behaves differently from its general category (for

example, a particular pizza variant associated with wine, while pizzas in general are associated with cola).

On the cost side, several problems remain. First, the method depends strongly on the taxonomy, which must be provided by the user. A poor or biased hierarchy directly produces poor or biased rules, and in many domains (e.g. user behavior logs, medical records) a clean is-a hierarchy is simply not available. Second, extending every transaction with its ancestors makes the search space much larger, because an item at depth d adds d extra symbols to each basket where it appears. This slows the algorithm and increases memory use compared with plain APRIORI. Third, the interest measure based on ancestor confidence only removes some redundancy. Rules at nearby levels often still overlap heavily, and the output can be hard to interpret. Sriphaew and Theeramunkong address this by mining only closed generalized itemsets, which keep the most specific frequent itemset in each equivalence class and drop the rest, giving a much smaller output without losing information ([7]). Fourth, a fixed is-a taxonomy cannot express richer relations between items, such as properties or part-of relations. Mansingh et al. replace the taxonomy with an ontology, which allows rules to be grouped or filtered by domain-level meaning rather than only by level ([5]). Finally, choosing good support and confidence thresholds is hard, because high and low taxonomy levels differ by orders of magnitude in frequency. A single threshold either buries the high-level rules in noise or makes the low-level rules invisible.

Overall, generalized association rules work well in domains with a clear and stable product hierarchy, such as retail or e-commerce, but are less useful when the hierarchy is unclear, incomplete, or changes often.

5 Numerical Prediction

5.1 Prediction target and data preparation

For the numerical prediction task, the target variable was defined as the subject-level average `mood` on the next day. Two different input representations were used for the two regression models.

Ridge regression was trained on the feature-engineered tabular dataset from Task 1C, which had likewise been used for the HistGradientBoosting classifier.

For the LSTM, the input was derived from the daily-aggregated dataset without additional handcrafted feature extraction. Each subject-day record contained daily summaries of the observed variables, and these records were arranged into short sequences. The temporal model thus received ordered daily observations directly, while the non-temporal model relied on manually engineered temporal summaries.

5.2 Compared regression algorithms

Ridge regression Ridge regression served as the regularised linear baseline. It was implemented using the `Ridge` class from `sklearn.linear_model` in `scikit-learn`. The feature-engineered predictors include numerous correlated lagged and summary variables, making ordinary least squares susceptible to instability from multicollinearity; the L_2 penalty mitigates this by shrinking coefficient estimates. The engineered features were supplied as a static predictor vector, so temporal structure entered the model solely through prior feature engineering. The regularisation parameter α was selected by grid search on the validation set, and the best-performing configuration was retained for final evaluation.

LSTM ensemble A Long Short-Term Memory (LSTM) network served as the temporal model, implemented in `PyTorch` using the `torch.nn` and `torch.utils.data` modules. Unlike Ridge regression, the LSTM was trained on the daily-aggregated dataset, receiving short sequences of daily observations and learning temporal dependencies directly from the ordered input.

The architecture comprised a single recurrent layer with 32 hidden units, followed by a dropout layer (rate 0.2) and a fully connected output layer. The model was optimised with Adam using MAE as the training loss; MSE and RMSE were additionally computed for evaluation. To reduce sensitivity to random initialisation, the model was trained multiple times with different random seeds and predictions were obtained by averaging the individual outputs.

5.3 Results

The two approaches yielded markedly different predictive performance. Ridge regression achieved a validation MAE of 0.292 and RMSE of 0.422; on the test set, it attained an MAE of 0.196 and RMSE of 0.280. The LSTM ensemble achieved a validation MAE of 0.428 and RMSE of 0.577, and a test MAE of 0.450 with RMSE of 0.632. See Table 8. The Ridge model thus substantially outperformed the temporal neural network on the final evaluation.

These results indicate that explicit feature engineering was highly effective at extracting predictive temporal structure for mood forecasting. Informative predictors summarising recent history proved more useful than requiring the LSTM to learn temporal dependencies directly from daily-aggregated sequences.

More broadly, the comparison demonstrates that an inherently temporal model does not automatically outperform a non-temporal one. Although LSTMs are well suited to sequential data in principle, their advantage may diminish when the dataset is modest in size or the temporal signal can be efficiently captured through handcrafted features. The superior generalisation of the regularised linear model suggests that the engineered tabular representation captured the dominant predictive relationships more effectively than the raw sequential input.

Table 8. Evaluation metrics for both models on validation and test sets.

Model	Val MAE	Val MSE	Test MAE	Test MSE
Ridge regression	0.292	0.178	0.196	0.079
LSTM ensemble	0.428	0.333	0.450	0.399

6 Evaluation

6.1 Characteristics of Evaluation Metrics

Let y_i denote the true value and $\hat{y}_i$ the predicted value for observation $i = 1, \ldots, n$. MSE and MAE are defined as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{2}$$

The metrics differ principally in their treatment of large errors. MSE squares the residuals, imposing disproportionate penalties on extreme deviations, whereas MAE grows linearly with residual magnitude and is therefore more robust to outliers. Consequently, MSE is preferred when large errors are particularly costly; MAE is favoured when interpretability and robustness are the primary concerns.

The two metrics yield identical model rankings when all residuals share the same absolute value, since squaring preserves order on non-negative numbers and both criteria then reduce to minimizing that common absolute error.

6.2 Impact of evaluation metrics

Both MAE and MSE were computed for Ridge regression and the LSTM ensemble. Ridge regression achieved a validation MAE of 0.292 and MSE of 0.178, and a test MAE of 0.196 and MSE of 0.079. The LSTM ensemble achieved a validation MAE of 0.428 and MSE of 0.333, and a test MAE of 0.450 and MSE of 0.399.

Both metrics yield the same qualitative conclusion: Ridge regression outperformed the LSTM ensemble on both validation and test sets. Nevertheless, they convey complementary information about model behaviour. MAE expresses average prediction error in the units of the target variable and is directly interpretable; a test MAE of 0.196 versus 0.450 indicates that Ridge predictions were, on average, substantially closer to observed mood values. MSE additionally captures the extent of large

individual errors: the considerably higher test MSE of the LSTM ensemble suggests its errors were not only larger on average but also more variable and extreme, whereas the lower MSE of Ridge regression reflects smaller and more stable errors. Reporting both metrics thus provides a more complete picture; their agreement here strengthens the evidence that Ridge regression was the more reliable predictor.

References

1. Toye, A., et al.: Benchmarking imputation methods for health time series. Journal (2025)
2. Cao, W., et al.: BRITS: Bidirectional recurrent imputation for time series. In: NeurIPS (2018)
3. Kazijevs, M., Samad, M.D.: Deep imputation of missing values in time series health data. J. Biomed. Inform. (2023)
4. Du, W., et al.: TSI-Bench: Benchmarking time series imputation. arXiv:2406.12747 (2024)
5. Bontempi, G., Ben Taieb, S., Le Borgne, Y.-A.: Machine learning strategies for time series forecasting. In: BISDM, LNBIP, vol. 138. Springer (2012)
6. Box, G.E.P., Jenkins, G.M., Reinsel, G.C., Ljung, G.M.: Time Series Analysis: Forecasting and Control, 5th edn. Wiley (2015)
7. Harvard Medical School: HMS – Harmful Brain Activity Classification. Kaggle (2024). `https://www.kaggle.com/competitions/hms-harmful-brain-activity-classification`
8. Team Sony: 1st place solution write-up. Kaggle (2024). `https://www.kaggle.com/competitions/hms-harmful-brain-activity-classification/discussion/492652`
9. Srikant, R., Agrawal, R.: Mining generalized association rules. Future Gener. Comput. Syst. **13**(2–3), 161–180 (1997)
10. Sriphaew, K., Theeramunkong, T.: Fast algorithm for finding generalized closed itemsets. In: Proceedings (2002)
11. Mansingh, G., et al.: Using ontologies to facilitate post-processing of association rules by domain experts. Inf. Sci. **181**(3), 419–434 (2011)